

Assign3

September 2, 2026

```
[1]: import pandas as pd
import numpy as np
```

```
[2]: import pandas as pd

data = {
    'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'Age': [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
    'Marks': [85, 78, 92, 88, 76, 90, 82, 95, 89, 80]
}

df = pd.DataFrame(data)

df
```

```
[2]:
```

	Name	Age	Marks
0	A	20	85
1	B	21	78
2	C	22	92
3	D	23	88
4	E	24	76
5	F	25	90
6	G	26	82
7	H	27	95
8	I	28	89
9	J	29	80

```
[3]: print(df)
```

	Name	Age	Marks
0	A	20	85
1	B	21	78
2	C	22	92
3	D	23	88
4	E	24	76
5	F	25	90
6	G	26	82
7	H	27	95

8	I	28	89
9	J	29	80

```
[4]: df.loc[[0, 4, 7, 8]]
```

```
[4]:
```

	Name	Age	Marks
0	A	20	85
4	E	24	76
7	H	27	95
8	I	28	89

```
[ ]: df.loc[3:7]
```

```
[6]: df.iloc[4:9, 2:5]
```

```
[6]:
```

	Marks
4	76
5	90
6	82
7	95
8	89

```
[7]: df.iloc[:, 1:4]
```

```
[7]:
```

	Age	Marks
0	20	85
1	21	78
2	22	92
3	23	88
4	24	76
5	25	90
6	26	82
7	27	95
8	28	89
9	29	80

```
[16]: import pandas as pd
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()

# Create DataFrame
iris_df = pd.DataFrame(
    iris.data,
    columns=[
        'SepalLengthCm',
```

```

        'SepalWidthCm',
        'PetalLengthCm',
        'PetalWidthCm'
    ]
)

# Add Species column
iris_df['Species'] = [
    iris.target_names[i] for i in iris.target
]

# Add Id column
iris_df.insert(0, 'Id', range(1, len(iris_df) + 1))

# Save as Iris.csv
iris_df.to_csv('Iris.csv', index=False)

print("Iris.csv created successfully!")

```

Iris.csv created successfully!

```

[17]: import pandas as pd

iris_df = pd.read_csv("Iris.csv")

```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	setosa
1	2	4.9	3.0	1.4	0.2	setosa
2	3	4.7	3.2	1.3	0.2	setosa
3	4	4.6	3.1	1.5	0.2	setosa
4	5	5.0	3.6	1.4	0.2	setosa

```

[18]: import pandas as pd

iris_df = pd.read_csv("Iris.csv")

```

```

[19]: # Delete row 4
iris_q5 = iris_df.drop(index=4)

# Delete column 3
iris_q5 = iris_q5.drop(columns=iris_q5.columns[3])

# Display the result
iris_q5

```

```

[19]:      Id  SepalLengthCm  SepalWidthCm  PetalWidthCm  Species
0      1           5.1           3.5           0.2      setosa

```

1	2	4.9	3.0	0.2	setosa
2	3	4.7	3.2	0.2	setosa
3	4	4.6	3.1	0.2	setosa
5	6	5.4	3.9	0.4	setosa
..	...	...	...	...	...
145	146	6.7	3.0	2.3	virginica
146	147	6.3	2.5	1.9	virginica
147	148	6.5	3.0	2.0	virginica
148	149	6.2	3.4	2.3	virginica
149	150	5.9	3.0	1.8	virginica

[149 rows x 5 columns]

```
[20]: import pandas as pd

employees = {
    'Employee_ID': [101, 102, 103, 104, 105],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Edward'],
    'Department': ['HR', 'IT', 'IT', 'Marketing', 'Sales'],
    'Age': [29, 34, 41, 28, 38],
    'Salary': [50000, 70000, 65000, 55000, 60000],
    'Years_of_Experience': [4, 8, 10, 3, 12],
    'Joining_Date': ['2020-03-15', '2017-07-19', '2013-06-01', '2021-02-10', '2010-11-25'],
    'Gender': ['Female', 'Male', 'Male', 'Female', 'Male'],
    'Bonus': [5000, 7000, 6000, 4500, 5000],
    'Rating': [4.5, 4.0, 3.8, 4.7, 3.5]
}

employees_df = pd.DataFrame(employees)

employees_df
```

```
[20]:
```

	Employee_ID	Name	Department	Age	Salary	Years_of_Experience	\
0	101	Alice	HR	29	50000	4	
1	102	Bob	IT	34	70000	8	
2	103	Charlie	IT	41	65000	10	
3	104	Diana	Marketing	28	55000	3	
4	105	Edward	Sales	38	60000	12	

	Joining_Date	Gender	Bonus	Rating
0	2020-03-15	Female	5000	4.5
1	2017-07-19	Male	7000	4.0
2	2013-06-01	Male	6000	3.8
3	2021-02-10	Female	4500	4.7
4	2010-11-25	Male	5000	3.5

```
[21]: print("Number of rows and columns:")
      print(employees_df.shape)
```

```
Number of rows and columns:
(5, 10)
```

```
[22]: employees_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Employee_ID           5 non-null     int64
1   Name                  5 non-null     object
2   Department            5 non-null     object
3   Age                   5 non-null     int64
4   Salary                5 non-null     int64
5   Years_of_Experience   5 non-null     int64
6   Joining_Date          5 non-null     object
7   Gender                5 non-null     object
8   Bonus                 5 non-null     int64
9   Rating                5 non-null     float64
dtypes: float64(1), int64(5), object(4)
memory usage: 532.0+ bytes
```

```
[23]: employees_df.describe()
```

```
[23]:
```

	Employee_ID	Age	Salary	Years_of_Experience	Bonus	\
count	5.000000	5.000000	5.000000	5.000000	5.0	
mean	103.000000	34.000000	60000.000000	7.400000	5500.0	
std	1.581139	5.612486	7905.69415	3.847077	1000.0	
min	101.000000	28.000000	50000.000000	3.000000	4500.0	
25%	102.000000	29.000000	55000.000000	4.000000	5000.0	
50%	103.000000	34.000000	60000.000000	8.000000	5000.0	
75%	104.000000	38.000000	65000.000000	10.000000	6000.0	
max	105.000000	41.000000	70000.000000	12.000000	7000.0	

	Rating
count	5.000000
mean	4.100000
std	0.494975
min	3.500000
25%	3.800000
50%	4.000000
75%	4.500000
max	4.700000

```
[24]: print("First 5 rows:")
      print(employees_df.head(5))

      print("\nLast 3 rows:")
      print(employees_df.tail(3))
```

First 5 rows:

	Employee_ID	Name	Department	Age	Salary	Years_of_Experience	\
0	101	Alice	HR	29	50000	4	
1	102	Bob	IT	34	70000	8	
2	103	Charlie	IT	41	65000	10	
3	104	Diana	Marketing	28	55000	3	
4	105	Edward	Sales	38	60000	12	

	Joining_Date	Gender	Bonus	Rating
0	2020-03-15	Female	5000	4.5
1	2017-07-19	Male	7000	4.0
2	2013-06-01	Male	6000	3.8
3	2021-02-10	Female	4500	4.7
4	2010-11-25	Male	5000	3.5

Last 3 rows:

	Employee_ID	Name	Department	Age	Salary	Years_of_Experience	\
2	103	Charlie	IT	41	65000	10	
3	104	Diana	Marketing	28	55000	3	
4	105	Edward	Sales	38	60000	12	

	Joining_Date	Gender	Bonus	Rating
2	2013-06-01	Male	6000	3.8
3	2021-02-10	Female	4500	4.7
4	2010-11-25	Male	5000	3.5

```
[25]: average_salary = employees_df['Salary'].mean()

      print("Average Salary:", average_salary)
```

Average Salary: 60000.0

```
[26]: total_bonus = employees_df['Bonus'].sum()

      print("Total Bonus:", total_bonus)
```

Total Bonus: 27500

```
[27]: youngest_age = employees_df['Age'].min()

      print("Youngest Employee's Age:", youngest_age)
```

Youngest Employee's Age: 28

```
[28]: highest_rating = employees_df['Rating'].max()

print("Highest Performance Rating:", highest_rating)
```

Highest Performance Rating: 4.7

```
[29]: sorted_df = employees_df.sort_values(by='Salary', ascending=False)

print(sorted_df)
```

	Employee_ID	Name	Department	Age	Salary	Years_of_Experience	\
1	102	Bob	IT	34	70000	8	
2	103	Charlie	IT	41	65000	10	
4	105	Edward	Sales	38	60000	12	
3	104	Diana	Marketing	28	55000	3	
0	101	Alice	HR	29	50000	4	

	Joining_Date	Gender	Bonus	Rating
1	2017-07-19	Male	7000	4.0
2	2013-06-01	Male	6000	3.8
4	2010-11-25	Male	5000	3.5
3	2021-02-10	Female	4500	4.7
0	2020-03-15	Female	5000	4.5

```
[30]: def performance_category(rating):
        if rating >= 4.5:
            return 'Excellent'
        elif rating >= 4.0:
            return 'Good'
        else:
            return 'Average'

employees_df['Performance_Category'] = employees_df['Rating'].
    ↪ apply(performance_category)

employees_df
```

```
[30]:
```

	Employee_ID	Name	Department	Age	Salary	Years_of_Experience	\
0	101	Alice	HR	29	50000	4	
1	102	Bob	IT	34	70000	8	
2	103	Charlie	IT	41	65000	10	
3	104	Diana	Marketing	28	55000	3	
4	105	Edward	Sales	38	60000	12	

	Joining_Date	Gender	Bonus	Rating	Performance_Category
0	2020-03-15	Female	5000	4.5	Excellent

1	2017-07-19	Male	7000	4.0	Good
2	2013-06-01	Male	6000	3.8	Average
3	2021-02-10	Female	4500	4.7	Excellent
4	2010-11-25	Male	5000	3.5	Average

```
[31]: print("Missing values in each column:")
      print(employees_df.isnull().sum())
```

```
Missing values in each column:
Employee_ID      0
Name             0
Department       0
Age              0
Salary           0
Years_of_Experience  0
Joining_Date     0
Gender           0
Bonus            0
Rating           0
Performance_Category  0
dtype: int64
```

```
[32]: employees_df = employees_df.rename(columns={'Employee_ID': 'ID'})

employees_df
```

```
[32]:
```

	ID	Name	Department	Age	Salary	Years_of_Experience	Joining_Date	\
0	101	Alice	HR	29	50000	4	2020-03-15	
1	102	Bob	IT	34	70000	8	2017-07-19	
2	103	Charlie	IT	41	65000	10	2013-06-01	
3	104	Diana	Marketing	28	55000	3	2021-02-10	
4	105	Edward	Sales	38	60000	12	2010-11-25	

	Gender	Bonus	Rating	Performance_Category
0	Female	5000	4.5	Excellent
1	Male	7000	4.0	Good
2	Male	6000	3.8	Average
3	Female	4500	4.7	Excellent
4	Male	5000	3.5	Average

```
[33]: more_than_5_years = employees_df[
      employees_df['Years_of_Experience'] > 5
      ]

print(more_than_5_years)
```

	ID	Name	Department	Age	Salary	Years_of_Experience	Joining_Date	\
1	102	Bob	IT	34	70000	8	2017-07-19	

2	103	Charlie	IT	41	65000	10	2013-06-01
4	105	Edward	Sales	38	60000	12	2010-11-25

	Gender	Bonus	Rating	Performance_Category
1	Male	7000	4.0	Good
2	Male	6000	3.8	Average
4	Male	5000	3.5	Average

```
[34]: it_employees = employees_df[
      employees_df['Department'] == 'IT'
    ]

print(it_employees)
```

	ID	Name	Department	Age	Salary	Years_of_Experience	Joining_Date	\
1	102	Bob	IT	34	70000	8	2017-07-19	
2	103	Charlie	IT	41	65000	10	2013-06-01	

	Gender	Bonus	Rating	Performance_Category
1	Male	7000	4.0	Good
2	Male	6000	3.8	Average

```
[35]: employees_df['Tax'] = employees_df['Salary'] * 0.10

employees_df
```

```
[35]:
```

	ID	Name	Department	Age	Salary	Years_of_Experience	Joining_Date	\
0	101	Alice	HR	29	50000	4	2020-03-15	
1	102	Bob	IT	34	70000	8	2017-07-19	
2	103	Charlie	IT	41	65000	10	2013-06-01	
3	104	Diana	Marketing	28	55000	3	2021-02-10	
4	105	Edward	Sales	38	60000	12	2010-11-25	

	Gender	Bonus	Rating	Performance_Category	Tax
0	Female	5000	4.5	Excellent	5000.0
1	Male	7000	4.0	Good	7000.0
2	Male	6000	3.8	Average	6500.0
3	Female	4500	4.7	Excellent	5500.0
4	Male	5000	3.5	Average	6000.0

```
[36]: employees_df.to_csv('modified_employees.csv', index=False)

print("Modified DataFrame saved successfully!")
```

Modified DataFrame saved successfully!

```
[ ]:
```